

TALLER – PROGRAMACIÓN ORIENTADA A OBJETOS

Diseño de Clases, Encapsulamiento y Herencia

Integrantes

- Máximo 2 estudiantes por grupo

Objetivo

Desarrollar un sistema orientado a objetos aplicando correctamente los conceptos de:

- Diseño de clases
- UML
- Encapsulamiento
- Herencia
- Constructores
- Métodos y validaciones

El taller busca que los estudiantes diseñen primero la solución y posteriormente la implementen en código usando C#.

Contexto del problema

Una empresa de tecnología desea desarrollar un sistema para gestionar diferentes tipos de dispositivos electrónicos.

Todos los dispositivos comparten información básica, pero dependiendo del tipo de dispositivo, pueden tener características y comportamientos específicos.

El sistema debe permitir registrar información general de los dispositivos y ejecutar diferentes acciones relacionadas con ellos.

Parte 1 – Diseño UML

Antes de iniciar la programación, el grupo deberá diseñar un diagrama UML de clases.

El diagrama debe incluir:

- Una clase padre que represente el concepto general de un dispositivo electrónico.
- Dos clases hijas que hereden de la clase principal.

Los grupos pueden escoger libremente los tipos de dispositivos que desean utilizar.

Ejemplos:

- Laptop
- Smartphone
- Tablet
- SmartWatch
- Consola de videojuegos

Requisitos del UML

La clase padre debe contener mínimo:

Atributos

- marca
- modelo
- precio

Métodos

- Encender()
- MostrarInformacion()

Las clases hijas deben:

- Heredar de la clase principal
- Tener mínimo un atributo propio
- Tener mínimo un método propio

Condiciones importantes

- La relación de herencia debe tener sentido lógico.
- No se deben repetir atributos heredados en las clases hijas.
- El diagrama debe representar correctamente la herencia utilizando la flecha UML correspondiente.

Parte 2 – Implementación en C#

Después de realizar el UML, el grupo deberá implementar el sistema en C#.

Encapsulamiento obligatorio

Todos los atributos del sistema deben estar protegidos utilizando: visibilidad private o protected.

No se permite el uso de atributos públicos.

Validaciones obligatorias

Marca

- No puede estar vacía.

Precio

- Debe ser mayor a 0.
- No puede superar los 50.000.000.

Modelo

- No puede estar vacío.

Constructores

Cada clase debe tener un constructor para inicializar sus atributos.

Métodos obligatorios

Clase padre

Debe contener:

- MostrarInformacion()
- Encender()

Clases hijas

Cada clase hija debe implementar mínimo:

- Un atributo propio
- Un método propio

Parte 3 – Uso del sistema

En el método Main (Program), el grupo deberá:

1. Crear mínimo un objeto de cada clase hija.
2. Probar métodos heredados.
3. Probar métodos propios.
4. Validar las restricciones solicitadas.
5. Mostrar información en consola.